AgentOS

Conditional Payment Escrow for Autonomous AI Agents

PRD v3.0 · Team1 India Speedrun JUNE 2026 · Theme: Agentic Payments · Protocols: x402 + ERC-8004 · Platform: Avalanche Fuji

x402 + ERC-8004	Conditional Escrow	Avalanche Fuji	Proof-of-Delivery Payments	14-Day Sprint

Executive Summary

x402 introduced machine-to-machine payments. ERC-8004 introduced on-chain agent identity. Both are now table stakes — every project entering the Agentic Payments space, including well-funded protocols like Kite, implements them.

AgentOS solves what none of them have solved: accountability for the output, not just the payment. When Agent A hires Agent B via x402 today, the payment is fire-and-forget — Agent B could return garbage and still keep the USDC. AgentOS introduces a Conditional Payment Escrow: USDC is locked in a smart contract, a verifiable output condition is agreed upon before the task, and payment only releases when the Research Agent submits proof of a qualifying delivery. If the condition fails, the payment returns. No human needed. Fully autonomous. Built natively on Avalanche.

> **"Where x402 says pay me first, AgentOS says prove it first — then you get paid."**

Why Existing Solutions Leave a Gap

The agentic payments space has credible, well-funded players. Understanding exactly what they cover — and what they miss — defines AgentOS's position.

Protocol / Product	What It Covers	What It Misses
Kite (GoKite.ai) $33M raised, own L1	Agent identity (Kite Passport), x402 payments, session-based spending limits, service marketplace, own chain	Requires human approval for every spending session.
Agentic.market (Coinbase)	Marketplace for x402-enabled services, agent discovery	No escrow, no output conditions, no ERC-8004 integratio

x402 raw (Coinbase protocol)	HTTP-native payment standard, USDC micropayments	Purely a payment protocol. No identity, no escrow, no ou
ERC-8004 standard	On-chain agent identity, capability registry, reputation scores	Identity only. No payment enforcement, no conditional lo
AgentOS	**x402 + ERC-8004 on Avalanche + Conditional Escrow + Proof-of-Delivery**	**Fills the accountability gap none of the above addres**

Problem

Autonomous agents transacting via x402 face a fundamental trust asymmetry. The paying agent has no guarantee that the receiving agent will deliver anything useful — and no recourse when it doesn't.

Pay-and-pray model	x402 requires payment before service. Once the USDC leaves, the payer has no leverage.
No output accountability	Reputation scores in ERC-8004 reflect completed transactions, not quality of output. An agent can build a high trust score by returning bad results that were still 'accepted'.
Human bottleneck	Kite solves spending limits — but requires humans to approve sessions. This kills true autonomy at scale, especially for fleets of agents.
No verifiable delivery primitive	There is no standard on-chain primitive for 'Agent B proved it did the work' before Agent B receives payment.

Vision

Make every agent payment accountable. If an agent cannot prove it delivered qualifying work, it does not get paid. No human approvals. No fire-and-forget. Every settled payment on AgentOS represents **verified output, not just completed transfer.**

AgentOS does not compete with Kite's infrastructure or Agentic.market's marketplace. It adds the **missing accountability primitive** that sits on top of x402 + ERC-8004 and makes autonomous agent commerce trustworthy — on Avalanche.

Core Innovation: Conditional Payment Escrow (CPE)

The Conditional Payment Escrow is a smart contract on Avalanche that replaces the direct x402 pay-first flow with a lock-verify-release cycle.

Standard x402 Flow

```
Personal Agent → HTTP call to Research Agent
Research Agent → HTTP 402: 'Pay 0.5 USDC'
Personal Agent → Sends 0.5 USDC (payment gone, no guarantee)
Research Agent → Returns result (could be anything)
```

AgentOS Conditional Escrow Flow

```
Personal Agent → Deploys CPE Contract, locks 0.5 USDC
Encodes condition: 'Output must be valid JSON
with field yield_opportunities[] containing 3+ entries'
Research Agent → Reads task from CPE Contract (ERC-8004 lookup)
Research Agent → Executes task, submits: output_hash + delivery_proof
CPE Contract → Evaluates condition on-chain
PASS → releases 0.5 USDC to Research Agent
FAIL → returns 0.5 USDC to Personal Agent
ERC-8004 → Updates reputation based on OUTCOME, not just payment
```

> Reputation in ERC-8004 now means something real: it reflects verified successful deliveries, not just transactions that happened. A high trust score becomes a genuine signal of quality.

Architecture — Three Components

01
CPE Smart Contract
Avalanche Fuji

- Locks USDC on task creation
- Stores encoded output condition (hash + condition type)
- Accepts delivery proof submission from Research Agent
- Evaluates condition on-chain (format check, field check, value threshold)
- Releases or returns payment autonomously
- Triggers ERC-8004 reputation write on settlement

02
Agent Task Protocol
Node.js / Python

- Personal Agent: creates task, encodes condition, deploys CPE
- Research Agent: discovers tasks via ERC-8004, reads condition, executes
- Research Agent: submits delivery proof (output hash + structured result)
- x402 modified: payment locked in CPE, not sent directly
- Both agents interact only with the CPE contract — no central server

03
Dashboard UI
React + TailwindCSS

- Condition builder: user defines task + verifiable output criteria
- Live escrow status: locked / pending proof / released / returned
- Delivery viewer: submitted output displayed with condition pass/fail
- Snowtrace links for every on-chain action (lock, proof, settlement)
- ERC-8004 reputation trail showing score before and after each task

Technical Specifications

CPE Contract — Key Functions

Function	Parameters	Action
`createTask()`	condition_hash, deadline, erc8004_addr	Locks USDC in contract, emits TaskCreated event
`submitDelivery()`	task_id, output_hash, result_uri	Research Agent submits proof; triggers condition evaluation
`evaluateCondition()`	task_id, output_data	On-chain check: format / field / value threshold; auto-settles
`getTaskStatus()`	task_id	Returns: LOCKED / PENDING_PROOF / SETTLED_PASS / SETTLE
`updateReputation()`	agent_addr, outcome	Writes to ERC-8004 registry: PASS increments, FAIL decrements

Condition Types (MVP)

For the hackathon, three condition types are supported:

Type	Example Condition	On-Chain Check
`FORMAT_JSON`	Output must be valid JSON	JSON.parse() equivalent in Solidity; revert if malformed
`FIELD_EXISTS`	Output must contain field 'yield_opportunities'	ABI-decoded struct check for required field key
`VALUE_THRESHOLD`	yield_opportunities array length >= 3	Array length comparison against stored threshold uint

Tech Stack

Layer	Technology
Smart Contract	Solidity 0.8.x · Hardhat · Avalanche Fuji C-Chain · OpenZeppelin ERC-20 SafeTransfer
Agent Runtime	Node.js · ethers.js · ERC-8004 ABI calls · USDC testnet
x402 Integration	HTTP 402 response handling · modified to escrow rather than direct transfer
Dashboard	React · TailwindCSS · ethers.js · Snowtrace API for live tx verification
Wallet	Hardhat test wallet (Research Agent) · MetaMask (Personal Agent demo)

MVP Scope — Hackathon (14 Days)

✓ INCLUDED	✗ EXCLUDED
● CPE contract deployed on Avalanche Fuji	● Complex AI-generated condition logic (NLP → condition encoder)
● Three condition types: FORMAT_JSON, FIELD_EXISTS, VALUE_THRESHOLD	● Multi-step milestone payments (single escrow only)
● USDC lock on task creation (real testnet USDC, not mocked)	● Cross-chain agent payments
● Research Agent submits delivery proof via contract call	● Dispute / arbitration mechanism
● On-chain condition evaluation → auto-settle (pass or fail)	● Agent marketplace or discovery UI
● ERC-8004 reputation update post-settlement (outcome-based)	● Subscription or streaming payment model
● Dashboard: condition builder + escrow status tracker	● DAO governance or token incentives
● Two-path live demo: successful delivery + failed delivery	● Enterprise fleet management
● All Snowtrace tx links live in the UI during demo	

Live Demo Scenario — Two-Path Walkthrough

The demo runs two paths in sequence. Path A is a successful delivery. Path B is a failed delivery with auto-return. Together, they show the full accountability loop. Total runtime: under 3 minutes.

PATH A — Successful Conditional Settlement

1

Condition Setup

User opens dashboard, sets task: "Find top 3 AVAX staking yields." Encodes condition: VALUE_THRESHOLD — yield_opportunities array must have >= 3 entries. Dashboard shows condition encoded and ready.

→ **Condition hash generated. Ready to lock.**

2 **USDC Locked in CPE Contract**

Personal Agent calls createTask() on CPE contract with 0.5 USDC. Dashboard shows escrow status: LOCKED. Snowtrace link to lock transaction appears live on screen.

→ **Tx: 0xabc...123 confirmed on Avalanche Fuji.**

3 **Research Agent Discovery**

Research Agent reads open tasks from CPE contract. Performs ERC-8004 lookup — confirms trust score 87/100, capability: research. Accepts task.

→ **ERC-8004 registry confirms agent identity on-chain.**

4 **Delivery Proof Submitted**

Research Agent executes query, returns JSON with 3 yield entries: Benqi 6.2%, GoGoPool 8.1%, AAVE v3 4.8%. Calls submitDelivery() with output_hash + result_uri. CPE contract evaluates condition.

→ **Condition: PASS. Array length = 3. Threshold met.**

5 **Auto-Settlement — Payment Released**

CPE contract auto-transfers 0.5 USDC to Research Agent. Dashboard shows escrow status: SETTLED_PASS. Second Snowtrace link shown — the settlement transaction.

→ **Tx: 0xdef...456 — 0.5 USDC released to Research Agent.**

6 **ERC-8004 Reputation Update**

CPE contract calls updateReputation() on ERC-8004 registry. Research Agent trust score: 87 → 88. Third Snowtrace tx link shown. Dashboard displays updated score.

→ **Reputation reflects verified delivery, not just payment.**

PATH B — Failed Delivery, Auto-Return

Immediately after Path A, run Path B to show the failure path. This is what makes AgentOS defensible — bad agents don't get paid.

1 **New Task, Same Condition**

Second task created. Same condition: JSON output with 3+ yield entries. 0.5 USDC locked in a new CPE escrow. This time, a low-trust Research Agent (score: 42) picks up the task.

→ **Escrow status: LOCKED.**

2 **Bad Delivery Submitted**

Low-trust agent submits: 'Here are some yields: AVAX is good.' (Plain text, not JSON, no structured data.) Calls submitDelivery() with this garbage output.

→ **CPE contract evaluates condition...**

3 Condition Fails — Payment Returned

Contract check: FORMAT_JSON fails. Output is not valid JSON. CPE auto-returns 0.5 USDC to Personal Agent wallet. Snowtrace link shows the return transaction.

→ **Condition: FAIL. Payment returned. Agent earned nothing.**

4 ERC-8004 Score Decremented

updateReputation() called with FAIL outcome. Low-trust agent score: 42 → 41. On-chain. Permanent. Dashboard shows updated score.

→ **Every failed delivery compounds the agent's reputation damage.**

> Judge moment: After Path B, point out that the low-trust agent's score is now 41. Ask: 'Would you hire an agent with score 41?' The answer is no — and now the ecosystem enforces that automatically.

User Roles

User	Research Agent Developer	Demo Builder (Hackathon)
Defines task and output condition via dashboard. Never approves a payment manually. Sees only condition setup and final result. Fully hands-off after condition is set.	Builds specialist agents that deliver structured, verifiable outputs. Registers on ERC-8004 with capability tags. Earns USDC per verified delivery. Incentivised to return quality work — bad output means no pay and score drop.	Plays both roles. Deploys CPE contract, deploys Research Agent backend, seeds ERC-8004 with two agents (trust score 87 and 42), runs both demo paths.

Competitive Positioning

Kite — Session-based spending You define a budget. Agent spends within it. Human approves each session. No output verification. Pay upfront, hope for results.	**AgentOS — Outcome-based escrow** Payment locked until output condition is met. No human approves sessions. Bad output = payment returned. Fully autonomous accountability.
x402 raw — Fire and forget You pay 402 response. You get a result. No guarantee on content or quality.	**AgentOS — Conditional x402** x402 triggers escrow lock. Payment only releases when delivery proof satisfies the encoded condition. x402 becomes accountable.

ERC-8004 reputation — Self-reported	AgentOS — Outcome-verified reputation
Reputation increments on transaction completion. An agent with 100 score could have delivered 100 garbage outputs that were never challenged.	ERC-8004 score only increments on VERIFIED successful delivery. Score now means actual quality, not just activity.

Success Metrics — Hackathon

CPE contract deployed	Verifiable on Snowtrace Fuji explorer	**✓ Live Tx Hash**
USDC locked in escrow	Real testnet USDC, not a balance variable	**✓ Live Tx Hash**
Condition evaluated on-chain	Solidity logic runs the check, not off-chain	**✓ Tx + Event Log**
Payment auto-settled	Both paths: release (pass) and return (fail)	**✓ 2 Live Tx Hashes**
ERC-8004 updated on outcome	Score changes post-settlement, outcome-based	**✓ Live Tx Hash**
Zero human approvals	No MetaMask pop-up during the demo flow	**✓ Demo observable**
Demo under 3 minutes	No live debugging. Both paths run clean.	**✓ Rehearsed**

Why This Scales

The Conditional Payment Escrow is a primitive — not a product. Once it exists on Avalanche, every x402-compatible agent ecosystem can plug in to get accountability. The scaling path is clear:

Short term	Condition library expands — LLM-graded output, API response codes, structured data schemas
Medium term	Multi-step milestone escrow — pay 20% on outline, 40% on draft, 40% on final
Long term	Enterprise fleet management — one policy vault with CPE enforcement for thousands of agents
Protocol play	CPE becomes an open standard on Avalanche — any agent marketplace can require CPE compliance